Project Proposal - Distributed Social Media Backend

Team: Ruiting Ma, Shuyi Zhang, Sihui Lyu

I. System Description & Scalability Requirements

Codebase Architecture

A X (Twitter) like social media platform decomposed into four containerized Go microservices:
User Service: Manages authentication, profiles, and follow/unfollow relationships
Tweet Service: Handles tweet creation & deletion, like & unlike operations, and asynchronous fan-out operations to populate follower timelines in Redis
Timeline Service: Serves materialized feeds from Redis cache with PostgreSQL fallback for cache misses
API Gateway: Routes requests, enforces rate limiting, and aggregates responses

Data Layer

PostgreSQL (primary + 2 or 3 read replicas): Source of truth for users, tweets, relationships, and like table with ACID guarantees
Redis cluster: Materialized timelines stored as ordered lists (LPUSH/LRANGE operations) with 1000-tweet cache per user, 24-hour TTL
(Optional) in Redis: Real-time like counters for viral tweets
(Optional) S3: Static assets (profile images, media uploads)

Scaling Mechanisms

The system must handle a 100:1 read-to-write ratio with the following scaling strategies:
Horizontal scaling: ECS Auto Scaling groups adjust service instance counts based on CPU/memory metrics
Read optimization: Fan-out-on-write pre-computes timelines in Redis cache; read replicas distribute database query load for cache misses and hydration queries
Stateless design: Services store no local state, enabling dynamic scaling without data loss
Asynchronous like aggregation: Decouples user-facing latency from batch count updates
Infrastructure as Code: Terraform manages multi-environment deployments with identical topologies

II. Experiments Plan

1. Impact of Redis Caching on Timeline Performance

Objective: Quantify the performance impact of Redis caching by comparing fan-out-on-write with Redis materialized timelines vs. without Redis.

Setup:
Configuration A (With Redis Cache): Tweet creation triggers async Redis writes (LPUSH) to all follower timelines. Timeline reads execute LRANGE on Redis lists, then hydrate tweet content from PostgreSQL replicas.
Configuration B (Without Redis Cache): Tweet creation only writes to PostgreSQL. Timeline reads execute SELECT * FROM tweets WHERE user_id IN (following_list) ORDER BY created_at DESC LIMIT 50 directly against PostgreSQL replicas with no caching layer.

Metrics:
Tweet creation latency (p50/p95/p99)
Timeline load latency (p50/p95/p99)
Database query rate (queries/sec on replicas)
Database CPU utilization (%)
Redis memory usage (Configuration A only)
Cache hit ratio (Configuration A only)

Load Profile:
10,000 concurrent users
90% timeline reads : 10% tweet writes
User follower distribution: 80% have <100 followers, 15% have 100-1000, 5% have 1000+

Expected Outcome:
Configuration A (With Redis) should achieve <50ms p95 timeline latency with minimal database load (only hydration queries)
Configuration B (Without Redis) should show 3-5× higher timeline latency (150-250ms p95) and 10× higher database query load
Quantify the cost-benefit tradeoff: Redis memory consumption (MB per 1000 users) vs. latency reduction and database offload
Identify breaking point: At what concurrency level does Configuration B saturate PostgreSQL replicas?

2. Eventual vs. Strong Consistency for Like Counts

Objective: Compare latency, throughput, and user experience tradeoffs between real-time (strongly consistent) and delayed (eventually consistent) like count updates.

Setup:
Baseline: Eventual Consistency
Like action: Write to likes table, return immediately (no count update)

Background worker: Batch aggregate counts every 30 seconds via UPDATE tweets SET like_count = …
Read path: Serve stale counts from replicas

Alternative: Strong Consistency
Like action: Write to likes table + synchronously execute UPDATE tweets SET like_count = like_count + 1
Read path: Serve real-time counts from replicas

Test Phases:
Phase 1 (Low Load): 1,000 concurrent users, 10 likes/sec, measuring latency, timeline load latency, and count accuracy, etc.
Phase 2 (High Load): 10,000 concurrent users, 500 likes/sec, measuring system throughput degradation, database lock contention for strong consistency, error rates, etc.
Phase 3 (Celebrity Spike): 10,000 users liking the same tweet within 10 seconds, measuring PostgreSQL primary CPU (write bottleneck), Redis memory usage, count convergence time, etc.

Metrics:
Like action response time & latency
Throughput
Database load
Timeline render time (does strong consistency slow reads?)

Load Profile:
70% timeline reads, 20% likes, 10% tweet writes
Like distribution: 80% of likes target 20% of tweets

Expected Outcome:
Eventual consistency should achieve low latency and much higher throughput.
Strong consistency should show bottleneck (PostgreSQL primary saturated) with latency

3. Failure Resilience Tests

Objective: Evaluate system behavior under replica failure and quantify read replica contribution to load distribution.

Test Scenarios:
Baseline (All Healthy): 3 PostgreSQL read replicas active, 10,000 concurrent users, steady-state load, measuring timeline latency and replica query distribution

Single Replica Failure: Terminate 1 read replica mid-test, observe results
Redis Cache Flush: Execute FLUSHALL on Redis mid-test to simulate cache failure, observe results

Metrics:
Mean time to recovery
Query redistribution latency
Error rates during failure windows
Cache hit rate recovery time

Expected Outcome:
System should tolerate single replica loss with acceptable latency increase.
Redis failure should expose database performance under full load.

Detailed Proposal & Updates

Distributed Social Media Backend

Team: Ruiting Ma, Shuyi Zhang, Sihui Lyu

I. Experiment Description

Impact of Redis Caching on Timeline Performance

This experiment evaluates the impact of Redis caching on read-heavy distributed systems by comparing two configurations of the same fan-out-on-write architecture: one with Redis materialized timelines and one without caching, forcing all reads directly to PostgreSQL.

Configuration A (With Redis Cache) implements the full fan-out-on-write pattern: when a user tweets, the system saves the tweet to PostgreSQL, looks up all followers, and asynchronously pushes the tweet ID to each follower's timeline cache using Redis LPUSH. Timeline reads execute a fast LRANGE operation on the pre-computed list (O(1) cache lookup), then hydrate full tweet content via a batch SELECT * FROM tweets WHERE id IN (...) query against PostgreSQL read replicas. This trades write amplification (O(n) Redis writes where n = follower count) for fast, database-light reads.

Configuration B (Without Redis Cache) uses the same fan-out-on-write logic for consistency, but disables Redis entirely. Timeline reads must execute complex SQL joins: SELECT t.* FROM tweets t JOIN follows f ON t.user_id = f.following_id WHERE f.follower_id = ? ORDER BY t.created_at DESC LIMIT 50. Every timeline load hits PostgreSQL with a query that scans the follows table and joins with tweets, creating O(m) database work where m = number of users followed. This configuration represents the baseline performance without caching infrastructure.

This experiment is important because it quantifies the value of caching infrastructure in read-heavy systems. While caching is a standard distributed systems pattern, this experiment provides concrete data on:

- Latency reduction: How much faster are cached reads compared to database queries?
- Database offload: How many queries per second can Redis absorb, preventing database saturation?
- Cost-benefit tradeoff: Is the operational complexity and memory cost of Redis justified by performance gains?
- Capacity planning: At what user concurrency does the no-cache configuration overwhelm PostgreSQL replicas?

Given social media's read-heavy nature (our system targets a 100:1 read-to-write ratio), this experiment validates the architectural decision to invest in Redis infrastructure. We use an 80/15/5 follower distribution as a baseline (80% of users with <100 followers, 15% with 100-1000, 5% with 1000+) to simulate realistic social network topology. Note: For users with millions of followers, fan-out-on-write creates write amplification (one tweet → millions of Redis writes), suggesting future work on hybrid architectures where celebrities bypass fan-out entirely, but this optimization is outside the scope of this experiment.

Eventual vs. Strong Consistency for Like Counts

This experiment compares latency, throughput, and user experience tradeoffs between strongly consistent (real-time) and eventually consistent (delayed) like count updates, directly applying CAP theorem principles to a practical scenario.

Strong consistency prioritizes consistency and partition tolerance (CP). The system guarantees accurate like counts, but under high write-contention such as thousands of users liking the same tweet simultaneously, row-level locking serializes writes and creates a bottleneck. Each like count operation must acquire a lock, wait for the previous write to complete, and then execute. Under extreme load, this may result in rejected requests or timeouts.

Eventual consistency prioritizes availability and partition tolerance (AP). The system always accepts like operations, writing to a likes table and returning immediately without updating the count. A background worker aggregates counts every 30 seconds via a single batch query. This solves write-contention by reducing thousands of lock-acquiring operations to one, though counts may be stale by up to 30 seconds.

The Celebrity Spike test (Phase 3) specifically targets the hotspot problem: 10,000 users liking the same tweet within 10 seconds. This simulates real-world viral content where a single database row becomes a write bottleneck. We expect strong consistency to show severe degradation such as high latency and potential request failures while eventual consistency maintains stable throughput. This experiment measures the real-world cost of consistency guarantees and informs decisions about where strong consistency is worth its cost versus where eventual consistency is acceptable.

Failure Resilience Tests

This experiment evaluates system behavior under component failure, testing fault tolerance, automatic failover, and graceful degradation which are critical properties for any production distributed system.

Replica Failure Test: We terminate one PostgreSQL read replica mid-test while the system is under load with 10,000 concurrent users. This tests whether the load balancer correctly detects the failure and redirects traffic to healthy replicas (automatic failover).
We measure:

- Query redistribution latency
- Error rate during the failure window
- Latency increase when operating with reduced replica capacity

Expected outcome:

System tolerates single replica loss with <25% latency increase and <1% error rate during the transition window.

Cache Failure Test: We execute FLUSHALL on Redis mid-test, simulating complete cache failure. This tests whether the system degrades gracefully to PostgreSQL or whether cache dependency causes cascading failure.
We measure:

- Cache hit rate recovery time
- Database query load increase
- Timeline load latency under full database dependency

Expected outcome:

System remains functional but with 3-5x latency increase. If PostgreSQL cannot handle the full load, we identify the capacity gap.

These tests connect directly to real-world operational concerns. AWS availability zone outages occur regularly, and any production social media system must tolerate component failures without complete service disruption. This experiment validates our architecture's resilience assumptions and identifies single points of failure before they cause production incidents.

II. Team Value Beyond AI

Design Judgment

Selecting these three experiments required prioritizing which distributed systems tradeoffs are most valuable to explore given course learning objectives. AI can generate arbitrary experiments, but choosing to focus on fan-out strategies, consistency models, and failure resilience requires understanding what concepts are central to CS6650 and how to test them meaningfully. We synthesized material across the course by connecting cache strategy to async processing patterns, consistency tradeoffs to CAP theorem, and failure resilience to graceful degradation principles into a coherent experimental design that builds on prior work.

Realistic Modeling

Load profile design required reasoning about real-world system behavior, not generating arbitrary numbers. The 80/15/5 follower distribution approximates power-law distributions observed in real social networks. The 100:1 read-to-write ratio reflects actual social media usage patterns. The celebrity spike scenario models documented viral content behavior. We treat these as sensitivity variables, testing alternative distributions to validate our assumptions which requires human judgment about what constitutes a valid test.

Interpretation and Debugging

When experimental results diverge from expectations, systematic troubleshooting is required. If eventual consistency shows worse throughput than strong consistency which is the opposite of expected, we would do the following actions. Check metrics including CPU utilization, database lock wait times, query execution plans. Isolate the problem layer by add timing logs to application code, verify index usage, reduce concurrent users to identify scaling thresholds. Form specific hypotheses such as batch aggregation query performs a sequential scan instead of using an index, or the 30 second batch window accumulates too many likes. This iterative hypothesis testing process requires human judgment, access to live system metrics, and the ability to adapt experimental design based on findings.

Operational Expertise

Deploying this system on AWS involves practical debugging that AI cannot execute autonomously. From prior assignments, we have experience resolving ARM versus AMD64 cross compilation issues on Apple Silicon, configuring VPC endpoints for private ECS subnets, debugging Terraform state inconsistencies, and troubleshooting container health check failures. These operational skills are essential for running valid experiments and cannot be replaced by AI generated infrastructure code alone.

III. Timeline

Project Milestones

Infrastructure: set up AWS resources (VPC, ECS, RDS, ElastiCache, etc.)
Mini services: build the four microservices with basic functionality
Data layer: configure PostgreSQL with replicas, Redis cluster
Experiment 1: cache strategy
Experiment 2: eventual vs strong consistency experiment
Experiment 3: failure resilience experiment
Analysis & report

Weekly Distribution

Week 1: Infrastructure + Core Service Skeleton (services deployable to ECS)
Week 2: Core Functionality (user auth, tweet CRUD, follow/unfollow, basic timeline working)
Week 3: Data Layer + Fan-out Implementation Redis caching, PostgreSQL replicas, fan-out-on-write with toggleable Redis on/off for experiments)
Week 4: Experiment 1 + Experiment 2 Setup
Week 5: Experiment 2 + Experiment 3
Week 6: Analysis + Report